

Comparative Analysis of Sequential Models for Mobile Network Traffic Forecasting

Author: **Ivan Manzi Ya Musana**

Dataset: Telecom Italia Milan CDR (SpazioDati / Big Data Challenge)

Models: SARIMA (SARIMAX + Fourier terms), LSTM, TCN

Project Links

Video presentation:

<https://www.awesomescreenshot.com/video/56692445?key=a9cf59a1652036828a7a7be31ac31d6a>

GitHub repository: https://github.com/Manzi453/Formative1_Techinique1

1. Introduction

Mobile network operators want to see short-term traffic swings coming so they can plan capacity, spot anomalies and shift radio and backhaul resources before users feel a problem. Traffic has strong daily and weekly rhythms, varies a lot between a business district and a residential street, and sometimes bursts due to an event or a fault. In this report I look at one-step-ahead forecasting for Internet traffic using the Telecom Italia “Milan Grid” CDR dataset [1]. It covers 10,000 grid squares in Milan, recorded every 10 minutes from 2013-11-01 to 2013-12-31.

The question I set out to answer is: how do different sequential models compare for one-step-ahead mobile network traffic forecasting, and how does their performance vary across geographical areas with different traffic characteristics? I was not trying to crown a single winner. I wanted to understand the signal, pick three models that really are different from each other, test them under the same protocol, and work out where and why each one succeeds or breaks. Everything is evaluated on the assignment week, December 16–22, 2013.

2. Related Work and Model Selection

SARIMA and its relatives are still solid baselines when traffic is strongly and stably periodic, because they model seasonality directly, use few parameters and are easy to interpret [2], [3]. Studies on cellular traffic do find, though, that ARIMA-type models get overtaken by neural sequence models as more history becomes available and the signal becomes bursty and nonlinear [4]. LSTMs are popular because their gated memory can pick up both short fluctuations and daily or weekly patterns without hand-built lag features [5], and Santos et al. showed on two months of very similar Milan data that recurrent networks beat naive and linear baselines, especially when demand is irregular [6]. Temporal Convolutional Networks (TCNs) are a third kind of model: stacks of dilated causal convolutions that read a whole window in parallel [7]. They reach long memory cheaply, but they squeeze the window into one fixed-size summary and have no built-in reason to care more about the latest step.

My own exploratory analysis (Section 4) pointed the same way. Total traffic is heavily skewed across squares, and the busiest squares combine a very regular daily cycle with a few sharp, short spikes. This mix of dominant seasonality plus nonlinear bursts is a good test for comparing a linear seasonal model against nonlinear learners, so I chose three models that differ in how they treat time. SARIMA with

Fourier seasonal terms is the interpretable linear baseline. LSTM is a recurrent model I expected to catch the bursty leftovers after seasonality is removed. TCN asks whether recurrence is really needed at all, or whether a parallel convolutional view of the same window is enough.

LSTM and TCN may look like close cousins because they read the same windowed input, but they are not. The LSTM updates its hidden state step by step, so the newest value is privileged by design. The TCN pools the whole window with no recency bias. This difference explains the failure I document in Section 6.3, where the two models fail for an identifiable architectural reason. Each model also has known weak spots: SARIMA assumes a fixed linear form, the LSTM needs more data and is opaque, and the TCN is cheap to train but lacks recency weighting.

3. Dataset and Data Preparation

Each daily file is a tab-separated, headerless table with one row per square, time interval and country code (square_id, timestamp_ms, country_code, SMS, call and internet fields). Because a square-interval pair is split across country codes, raw rows are about 20–30 times more numerous than what I need. I use only the internet column. The daily files are 300–400 MB each, and the two months I use (2013-11-01 to 2013-12-31) come from an archive of 62 files, about 20 GB.

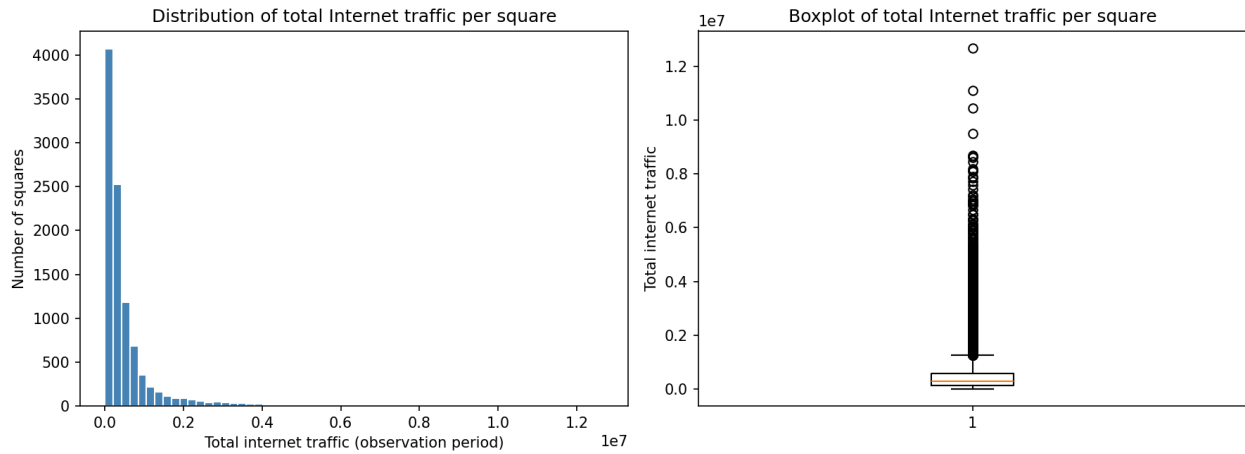
Memory. My machine has 8 GB of RAM. Reading one day with default pandas settings already peaks near 635 MB for about 4.84 million rows, so two months would need tens of gigabytes. I built a two-pass pipeline instead: read only the needed columns, downcast to int32 and float32, stream each file in 2,000,000-row chunks, and collapse country codes immediately with a groupby. The first pass only accumulates total traffic per square to rank them. The second pass re-reads the archive keeping only the five squares of interest. On one day this reduces peak memory from about 635 MB to 292 MB (roughly 54%), and the full archive finishes in a few minutes [8]. The cost is reading the data a second time, and the technique wouldn't scale to citywide modelling that needs all 10,000 series at once, but it works well for single-area forecasting on a laptop.

One caveat. Files are selected by the date in their filename, but timestamps are internal in UTC whereas Milan is UTC+1, so the file for 2013-11-01 actually starts at 2013-10-31 23:00. This pushes the window an hour at each end (well under 0.1% of the data) and doesn't affect the ranking, but I'd rather mention it than bury it. Missing 10-minute intervals are rare and interpolated with linear interpolation; model-specific preprocessing is described in Section 5.

4. Exploratory Analysis

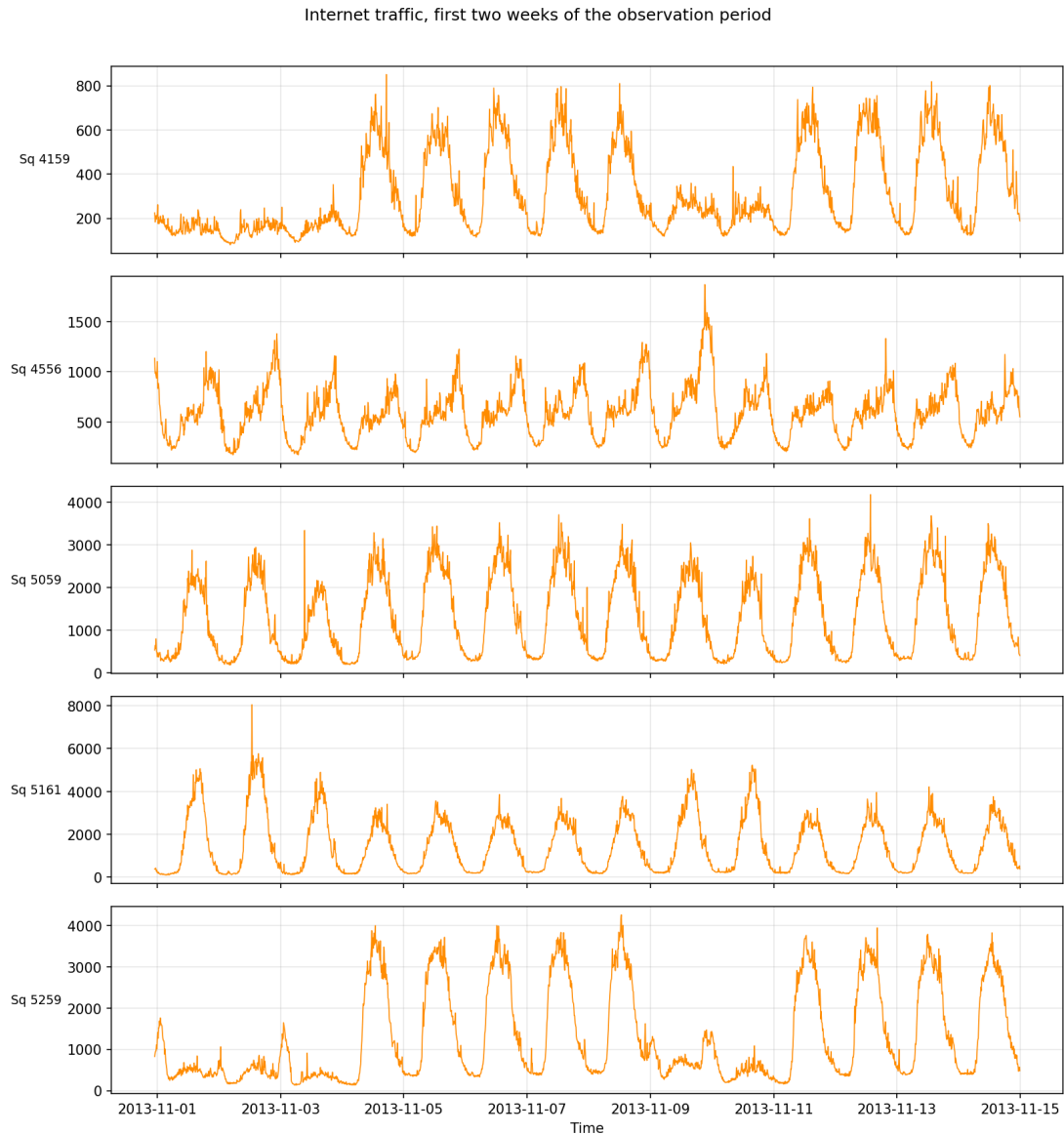
Total Internet traffic per square is considerably right-skewed (skewness about 4.28, excess kurtosis about 25.7, mean of 549,973 against a median of 274,706). The top 1% of squares carry about 11.1% of all traffic, which fits with a picture of a few business and transit hubs dominating a long tail of quieter residential squares (Figure 1). The three busiest squares are 5161, 5059 and 5259 (12.68M, 11.09M and 10.44M). A one-month provisional ranking gave the same three, so the ranking is stable. I also consider squares 4159 and 4556, requested in the brief (Figure 2).

Figure 1. Distribution of total Internet traffic per square over the two-month period (10,000 squares).



All five squares have a daily rhythm, with a trough around 04:00-06:00 and a broad daylight peak, but they vary in scale and shape. Square 4159 drops off clearly at weekends (Nov 9-10) and is like a commuter-driven office area. Square 4556 is noisier with a higher overnight baseline and is more like a residential or mixed-use area. Squares 5059, 5161 and 5259 have the largest peaks and move together, suggesting they are neighbouring. Square 5161 has the most volatility, with two short spikes on Sundays (Nov 3 and Nov 10) about two to three times their typical Sunday peak. It's a natural stress test for sudden, non-repeating behaviour.

Figure 2. Internet traffic in the five squares over the first two weeks of the observation period.



For square 5161 I also check its stationarity and autocorrelation (Figures 3 and 4). The ACF (Figure 4) peaks at lags of about 144 and 288 steps, one and two days, classic sign of strong daily seasonality, while the PACF cuts off after one or two lags. Once the daily cycle is incorporated the leftover short-term dependence is low-order. That supports a low-order autoregressive term plus an explicit seasonal term in SARIMA, and a window of at least 144 steps for the LSTM and TCN. The STL decomposition (Figure 3) shows a large stable seasonal part, a slowly moving trend, and residuals that are small except on the anomalous days. The Augmented Dickey-Fuller test rejects a unit root on the raw series (statistic about -19.13 , p near 0) and on the first difference (about -15.05). ADF speaks to trends and not to seasonality, so differencing is not strictly needed, but the daily cycle still needs to be modelled on purpose.

Figure 3. STL decomposition (period = 144, i.e. 1 day) for square 5161.

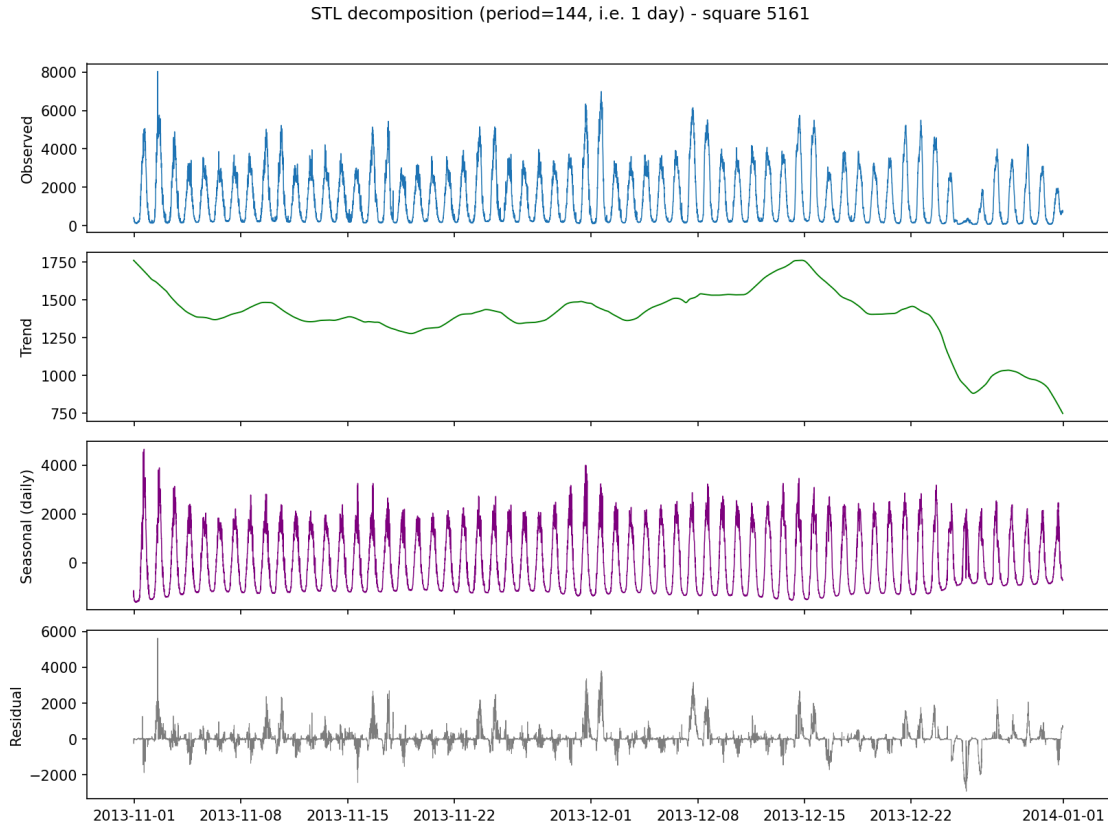
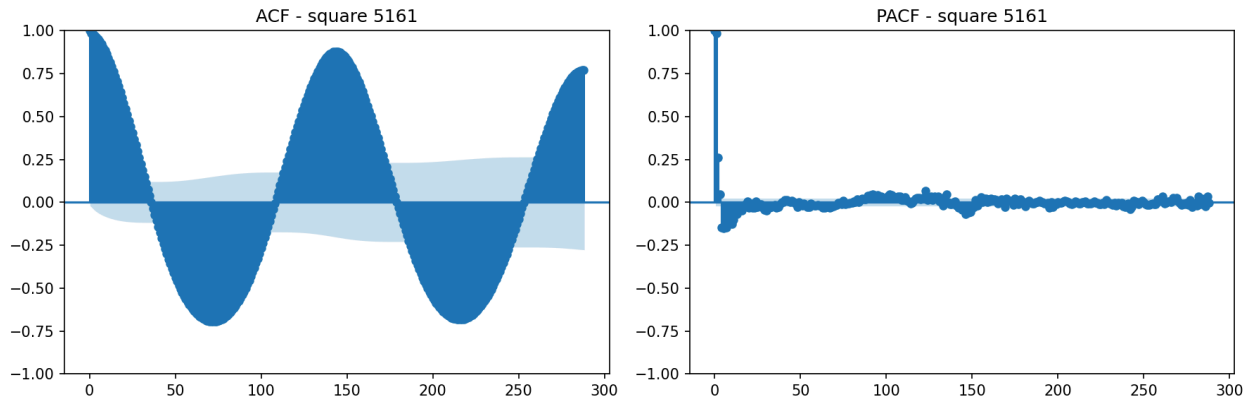


Figure 4. ACF and PACF for square 5161.



5. Methodology

Task and split. At each 10-minute step t of the evaluation week a model is given the history up to t and predicts $t+1$. Evaluation is walk-forward using true observed values, never the model's own earlier predictions, so all models get a fair one-step error. For each target square I train on 2013-11-01 00:00 to 2013-12-15 23:50 (45 days) and test on 2013-12-16 00:00 to 2013-12-22 23:50 (7 days, 1,008 points).

SARIMA with Fourier terms. The input is the univariate series. SARIMA has no fixed window: its Kalman filter carries the whole history forward. I represent seasonality with sine and cosine terms at the 144-step daily period, because a full seasonal order at that period is impractical to fit and tune [2]. Order and number of harmonics comes from grid search. No scaling is used, and during evaluation the state is

updated with each true value without re-fitting coefficients. I always enforce stationarity and invertibility, because without them the optimiser can drift to explosive coefficients and the forecasts diverge.

LSTM. The input is a sliding window of min-max-scaled values, with the scaler fitted on training data only to avoid leakage. The network is two stacked LSTM layers with dropout and one dense output, trained with Adam. The last 3 days of training are held out for validation, and early stopping (patience 5, best weights restored) is active in the final fit, as in tuning. Test predictions use the true preceding values, in line with walk-forward evaluation.

TCN. Same window, scaling and early stopping as the LSTM. It has six residual blocks of dilated causal convolutions (dilations 1 to 32, kernel size 3) with dropout, giving a receptive field of about 253 steps. A GlobalAveragePooling layer averages the learned representation across the window before the single output unit, so every position gets equal weight by construction. That detail matters in Section 6.3.

Tuning. Because three different models made manual tuning impractical, I ran a grid search once on square 5161, with the last 3 days of training as validation so the test week is untouched. The grids were SARIMA (p,d,q) in {(1,1,1), (2,1,2), (3,1,1)} with 0, 2 or 4 harmonics; LSTM with sequence length 72 or 144, units [32] or [64,32], and learning rate 0.001 or 0.0005; and TCN with sequence length 72 or 144, 16 or 32 filters and the same learning rates. The winning neural models were retrained for up to 40 epochs. The TCN architecture itself was fixed by design.

6. Results and Discussion

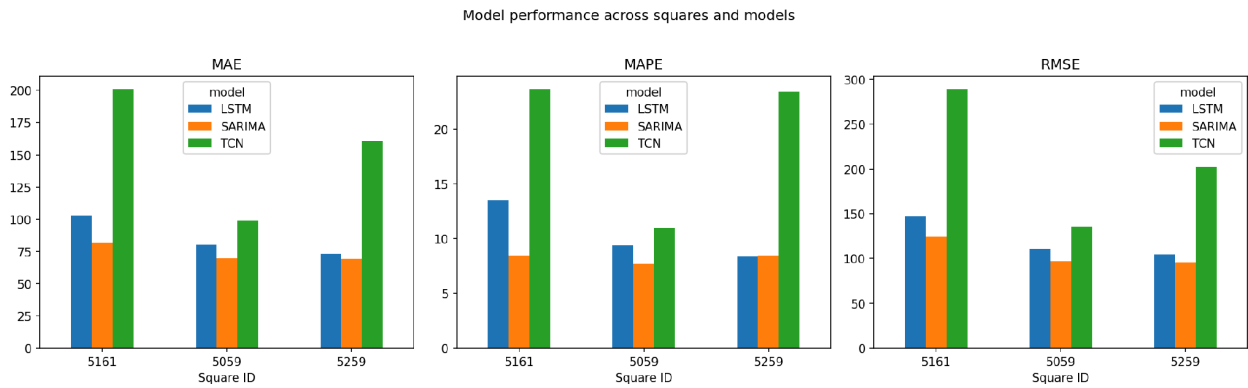
Tuning outcome. The search picked SARIMA (2,1,2) with 4 harmonic pairs, an LSTM with window 72 and a single 32-unit layer, and a TCN with window 144 and 32 filters. Adding harmonics cut SARIMA's validation RMSE from about 184 to 154, while the (p,d,q) order barely mattered (153–154), which matches the PACF. For the LSTM, the simplest setup won clearly (RMSE about 200.5 against 220–308 for the others).

Per-square performance. All runs used a MacBook Air (Apple M2, 8 GB, CPU only) with Python 3.11, and timings are from a single run per model and square, so treat them as approximate. Table 1 and Figure 5 show the accuracy results.

Table 1. One-step-ahead results on the evaluation week (SARIMA in bold).

Square	Model	MAE	MAPE (%)	RMSE
5161 (top-1)	SARIMA	81.91	8.44	123.84
5161	LSTM	103.04	13.52	147.58
5161	TCN	201.04	23.65	289.20
5059 (top-2)	SARIMA	69.73	7.69	96.75
5059	LSTM	80.34	9.37	110.78
5059	TCN	99.04	10.99	135.56
5259 (top-3)	SARIMA	69.15	8.46	95.94
5259	LSTM	73.16	8.38	104.49
5259	TCN	160.85	23.41	202.02

Figure 5. Model performance across squares and models.



SARIMA has the lowest MAE and RMSE on every square. LSTM is second on those metrics everywhere and clearly beats TCN on every square and metric. The only exception is square 5259, where LSTM's MAPE (8.38) is just under SARIMA's (8.46), which I read as a tie. TCN is the weakest model throughout, and on the volatile square 5161 its RMSE (289.20) is more than double SARIMA's (123.84).

Timing is a slightly surprising story. The LSTM trained faster than the TCN on every square (about 20–26 s against 37–81 s), the opposite of what the architecture suggests, because its validation loss levelled off sooner and early stopping kicked in earlier. Inference is near-instant for both networks. SARIMA trains fastest (2–14 s) but is the slowest at prediction time (63–71 ms per step, about a minute for the week) because of the Kalman update at each step.

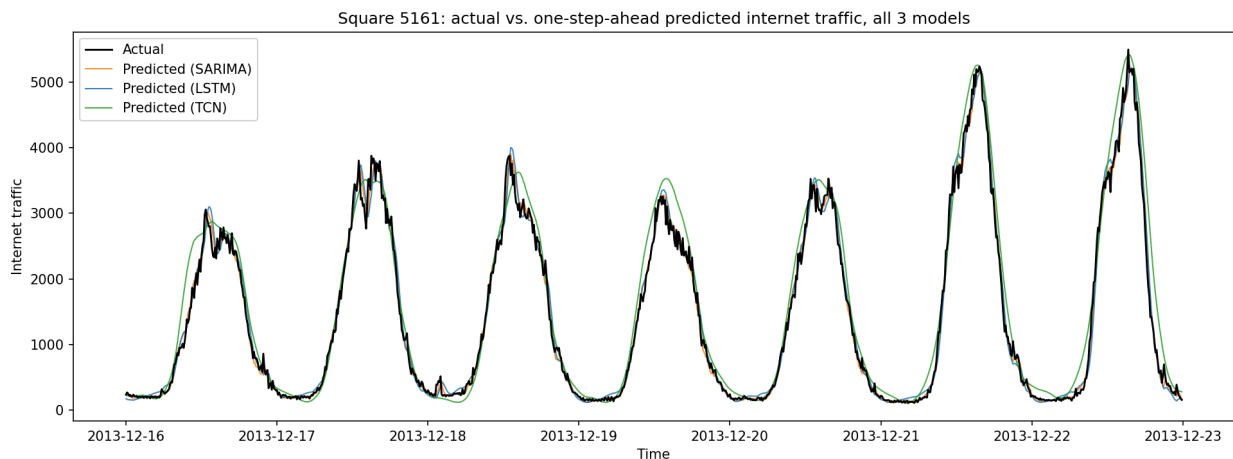
The reason SARIMA does so well is in the data. The signal is a strong, stable daily cycle with low-order leftover dependence, close to the best case for a linear model: it is handed the daily cycle directly. The LSTM, with 45 days of history and validation-gated stopping, closes most of the gap. The TCN saw exactly the same data and stopping rule, so its larger error points at the architecture and not at data volume.

I should also be honest about one expectation that did not hold. In Section 2 I expected the LSTM's recurrence to help most on the volatile square 5161. It did not. It came closest to SARIMA on the calm square 5259 and still trailed on 5161 (RMSE 147.58 against 123.84). Two isolated Sunday spikes in two months are probably too few examples to learn from, so the LSTM likely treated them as noise. That gap between my original reasoning and the result is not fully closed by these experiments.

6.3 Failure case: TCN under-tracks fast changes

The clearest failure is on square 5161, where the TCN's RMSE is 133% higher than SARIMA's, the largest relative gap (40% on 5059 and 111% on 5259). Figure 6 shows why. SARIMA and LSTM follow the actual series closely, including the sharp secondary peak on Dec 20, while the TCN's forecast is smoother and phase-shifted. It rounds off sharp peaks (Dec 16 and Dec 22, about 5,000–5,500) and overshoots in the troughs (around Dec 16–17 and Dec 19).

Figure 6. Square 5161: one-step-ahead predictions of all three models against the actual series, evaluation week.



The same behaviour shows up on square 5259, most clearly in an irregular weekend dip around Dec 21–22, and mildly on 5059, in proportion to each square’s gap. That suggests a general property of the architecture and not a quirk of one square. The likely cause is the pooling head: it compresses the 144-step window into one vector and weights every position equally. SARIMA conditions on the latest true observation at each step, and the LSTM’s hidden state naturally emphasises recent inputs. Since the gap remained with identical data and stopping rules, “the TCN just needed more data” is ruled out. In practice, I would not deploy this TCN as-is on volatile squares. The first fix to try is replacing or supplementing the pooling with something that privileges recent timesteps, such as a final recurrent or attention layer, or using the last-timestep vector.

7. Conclusion and Future Work

I compared SARIMA with Fourier terms, an LSTM and a TCN for one-step-ahead Internet-traffic forecasting on the Milan CDR data, using the three busiest squares over December 16–22, 2013. Three things stand out. First, once the daily seasonality found in the EDA was given to SARIMA as a Fourier basis, it had the lowest MAE and RMSE on all three squares, and LSTM only matched it on MAPE for the calmest square, so a well-specified linear model is still a strong baseline. Second, the LSTM closed most of the gap with enough data and validation-gated training, and its simplest configuration (one 32-unit layer, 72-step window) generalised best and beat the TCN everywhere. Third, the TCN’s larger error was not a data problem: its smoothed, phase-shifted forecasts follow from the missing recency weighting in its pooling head.

There is room to improve each model: per-square automatic order search and periodic re-fitting for SARIMA, a wider architecture search (for example attention) for the LSTM, and a last-timestep or attention head for the TCN. The main limitations are that hyperparameters were tuned on one square only (5161) and reused, the TCN architecture was fixed by design, walk-forward SARIMA never re-estimates coefficients, SARIMA needs its stationarity safeguard for safe unattended use, and the UTC offset shifts the ranking window by about an hour. Future work could cover low-traffic residential squares, a modified TCN head, cross-square spatial features (neighbouring squares moved together), and a hybrid where a neural model corrects SARIMA’s residuals.

References

- [1] G. Barlacchi et al., “A multi-source dataset of urban life in the city of Milan and the Province of Trentino,” *Scientific Data*, vol. 2, p. 150055, 2015.
- [2] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 2nd ed. OTexts, 2018.
- [3] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed. Wiley, 2015.
- [4] A. Azari, P. Papapetrou, S. Denic, and G. Peters, “Cellular Traffic Prediction and Classification: A Comparative Evaluation of LSTM and ARIMA,” in *Discovery Science (DS 2019)*, LNCS 11828, Springer, 2019, pp. 129–144.
- [5] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [6] G. L. Santos et al., “Predicting short-term mobile Internet traffic from Internet activity using recurrent neural networks,” *Int. J. Network Management*, 2021.
- [7] S. Bai, J. Z. Kolter, and V. Koltun, “An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling,” *arXiv:1803.01271*, 2018.
- [8] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proc. 9th Python in Science Conf.*, 2010, pp. 56–61.